SIEM LAB · VOLUME 2

Hardening Windows Server & Validating Your Detection

Secure a Windows Server 2022 VM from Kali, enroll it in your Wazuh SIEM, and prove your SIEM actually catches what happens next.

Written for	A Linux-only learner building their first security lab
Target machine	WinServer — Windows Server 2022 (build 20348), 4 GB RAM
Assessment host	Kali 10.0.0.30 · SIEM-Ubuntu 10.0.0.40 (Wazuh 4.14.8)
Lab network	VirtualBox `intnet`, 10.0.0.0/24, fully isolated
Assumes	Volume 1 — you can already operate the Wazuh dashboard
Document date	30 September 2026

> **What this guide is.** It does three things, in order: it *audits* your Windows Server from Kali, it *hardens* it, and it then deliberately generates suspicious activity *on your own server* so you can prove Wazuh detects it. That last part is the whole point — a detection you have never seen fire is not a detection, it is a hope.

Scope — read this before you start

Being precise about this is not bureaucracy. It is the professional habit, and it is why penetration testers get to do interesting work legally.

Everything in this guide targets one machine you own

Every command here runs against `10.0.0.50` — a VirtualBox VM on your own laptop, on an isolated network segment with no route to anything. There is no third-party system here, no wordlist, and no credential attack of any kind. Where authentication events are needed for a detection test, you use a throwaway account you created yourself and a single password you deliberately mistype, so that log events exist to be detected. You are generating log volume to test a sensor, not attempting to access anything.

This guide covers	This guide does not cover
• Reading a Windows Server's configuration — accounts, password policy, SMB, auditing, firewall — from both sides • Fixing what is wrong, with the reasoning behind each setting • Enrolling Windows into an existing SIEM correctly • Generating controlled activity on your own host and confirming the SIEM reports it • Writing Wazuh rules for Windows event IDs • Writing it all up	• Compromising, exploiting or gaining unauthorised access to any system • Password spraying, brute forcing, or use of wordlists and cracking tools • Exploit payloads, weaponised tooling, or post-exploitation • Anything against a machine you do not own and have written authorisation for

The habit worth building now, not later

Professional practice is that a system is "in scope" only when someone has documented permission for it. For your own lab that is trivially true — but write the scope line down anyway when you start your notes in Part P8. It is the first thing a hiring manager looks for in a portfolio, and it is the thing that keeps hobbyists out of legal trouble.

Contents

P0 Windows for a Linux person

P1 Prepare WinServer in the lab

P2 Enroll it in Wazuh

P3 Audit it from Kali (read-only)

P4 Harden it — the main event

P5 Validate your detection

P6 Windows event coverage and Sysmon

P7 Detection rules for Windows

P8 The writeup template

P9 Project roadmap — what to build next

A Windows security event ID reference

B Command cheat sheet

P0 Windows for a Linux person

You will be much faster if this section is read rather than skipped. Every concept here trips up Linux people, and all of them matter for the work ahead.

P0.1 The mapping table

Linux concept you know	Windows equivalent	What will confuse you
Systemd service	**Windows Service**, managed by the SCM	Services are configured in `services.msc`, not `systemctl`. Some can be set to start automatically via PowerShell (Part P2.3).
`/var/log/auth.log`	**Event Log** — structured records with numeric IDs	The single most important difference. Windows does not have text log files you `grep`. Everything is an event with a numeric ID, and **the ID is what matters**. See Appendix A.
`/etc/passwd`, `/etc/shadow`	SAM database, NTDS on a domain controller	Not plain text. Never edit it directly.
netplan / NetworkManager	`ncpa.cpl` or PowerShell NetTCPIP	Adapters are named by localised display name ("Ethernet", "Ethernet 2"), localised. If you changed the OS language, the name changes too. Prefer matching on the interface index or description where you can.
Users and groups in `/etc/group`	Local users / groups, plus Active Directory on a DC	Two completely separate systems. A standalone server has only local accounts.
Config files in `/etc`	**Registry**	One enormous tree of settings. `HKLM\SYSTEM\CurrentControlSet\Control\Lsa` is the one you will use most. Edit with `regedit` or `reg add`.
iptables / nftables	Windows Defender Firewall	Profiles (Domain / Private / Public) decide which rule set applies. Getting the profile wrong is why "my firewall rule doesn't work".
bash, shell scripts	**PowerShell**	Object-based, not text-based. Where Linux pipes strings, PowerShell pipes rich objects. This actually makes many admin tasks easier — but it is a different mindset.
systemd targets / boot order	Windows services + dependencies	"Startup type" is the equivalent of an enable/disable.

P0.2 Two facts about Windows security you cannot skip

> **1. Windows denies by default, Linux allows by default**
>
> An empty firewall ruleset on Windows blocks everything inbound. This will bite you the very first time: the Wazuh agent ships, starts, and then sends *nothing*, because Windows Firewall silently drops inbound port 1514. You will lose an afternoon to this if nobody tells you. Part P2.4 has the fix.

2. Event ID 4624 is logged by default; the useful ones are not

A default Windows Server logs successful and failed logons and little else. No process creation. No detailed file access. No privilege changes. **Part P4.4 turns on the auditing your SIEM needs** — and that section is the single highest-value part of this entire guide, because without it Wazuh will be nearly blind on Windows.

P0.3 PowerShell survival skills

You are about to run a lot of PowerShell. It is not hard, but it is strict about punctuation.

Need	Command
Run as Administrator	`Windows Terminal` → dropdown → *Administrator*. Or `Win` + `X` → *Terminal (Admin)*
Find a command	`Get-Command *smb*`
Explain a parameter	`Get-Help Set-SmbServerConfiguration -Full`
See current state (no changes)	`Get-SmbServerConfiguration`
Apply a change	`Set-SmbServerConfiguration ...` — the pattern is always Get to read, Set to change
Output as table / list	`... | Format-Table -AutoSize` `... | Format-List`
Continue past an error	`$ErrorActionPreference = 'Continue'`

Get-Command and Get-Help are your man pages

Same idea as `man` in Linux. PowerShell ships with real help on most modules, including examples. `Get-Help about_Firewall` is the equivalent of reading the admin guide, and it is genuinely good.

P1 Prepare WinServer in the lab

P1.1 Current state, and the problems it causes

WinServer currently has a single adapter on `intnet` and no internet connection. That is fine for isolation, and it is a problem for installing an agent. Also worth noting from the VM config: the install ISO is still attached to the DVD drive.

Current	Worth keeping?	Consequence
NIC 1 — intnet only	Yes	This is the lab network. Keep it. Everything agent-related happens here.
NIC 2 — *disabled*	Deliberate or accidental	**No internet.** Cannot download the agent, cannot run Windows Update, cannot install tools. Fix in P1.3.
Install ISO attached to DVD	No	Boots into setup if boot order is ever changed. Detach it.
No static IP	—	There is no DHCP server on `intnet`, so this adapter likely has no IPv4 address at all — the same failure you already fixed twice on Ubuntu. See P1.4.
Guest Additions 7.2.12	Yes	Gives you an out-of-band command channel. Your safety net. Do not uninstall.
4 GB RAM	Acceptable	Tight but workable for Server 2022. 6 GB is more comfortable if the host has headroom.

P1.2 Snapshot first. Always.

> **Do this before any other step**
>
> Power off WinServer in VirtualBox and take a snapshot named `winserver-clean-30sep`. Everything from here on is reversible in about a minute, which turns every remaining risk into a non-event.

P1.3 Give it internet, or commit to air-gap — your choice

Option A — add a NAT adapter (recommended)

Same pattern you used for Kali yesterday: keep `intnet` for lab traffic, add NAT purely for internet. Requires the VM powered off.

```
VBoxManage modifyvm "WinServer" --nic1 none
VBoxManage modifyvm "WinServer" --nic2 intnet --cable2 on
VBoxManage modifyvm "WinServer" --nic3 nat   --cable3 on
VBoxManage modifyvm "WinServer" --natpf3 "winrm,tcp,127.0.0.1,15985,,5985"
VBoxManage startvm "WinServer" --type headless
```

> **Get the adapter slot number right — you have been here before**
>
> `--natpf1` , `--natpf2` , `--natpf3` refer to the **adapter slot number**, not to "the NAT one". Above I moved intnet to slot 2 and NAT to slot 3, so the forward uses `--natpf3` . Using the wrong number writes the rule into a disabled block, reports *no error at all*, and does nothing. Always verify:
>
> ```
> VBoxManage showvminfo "WinServer" | Select-String "NIC|Rule"
> ```
>
> Simpler alternative: keep intnet in slot 1, add NAT in slot 2, and use `--nic2 nat --natpf2` exactly as you did for Kali. Either is fine — just match the number to the slot.

Option B — stay air-gapped and sideload the agent

More realistic for a hardened server, and it forces you to learn offline installation — a genuinely useful skill. No adapter changes at all.

On the Windows host, download the agent installer once:

```
# PowerShell on your Windows 11 host
curl.exe -o "$env:USERPROFILE\Downloads\wazuh-agent-4.14.8-1.msi" `
  https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.8-1.msi
```

Share that folder with the VM through VirtualBox — run this on the *host* shell:

```
VBoxManage sharedfolder add "WinServer" "wazuhmsi" `
  --hostpath "C:\Users\pc\Downloads"
```

Then inside the VM, from an Administrator PowerShell. Guest Additions maps the share to `\\tsclient\wazuhmsi` :

```
msiexec.exe /i "\\tsclient\wazuhmsi\wazuh-agent-4.14.8-1.msi" /q `
  WAZUH_MANAGER="10.0.0.40" WAZUH_AGENT_NAME="winsrv-01" `
  WAZUH_AGENT_GROUP="windows,servers"
```

Verify the share is visible from inside the VM before trying the install:

```
Get-ChildItem "\\tsclient\wazuhmsi" | Select-Object -First 5 Name, Length
```

Which to pick

	Option A (NAT)	Option B (air-gap)
Ease of lab use	Much easier — you can install anything	Every tool needs sideloading
Realism	Low — real servers rarely have both	High — matches production segmentation
Skills gained	Networking, Windows Update	Offline deployment, which is what you would actually do at scale
Recommendation	Do this one first	Try it for the hardening phase, when you want a clean, quiet system

P1.4 Detach the ISO and set a static address

Detach the install media

```
VBoxManage modifyvm "WinServer" --medium none
VBoxManage showvminfo "WinServer" | Select-String "DVD|ISO"
```

Should now show no DVD image attached. This prevents a boot-into-setup surprise later.

Set the lab IP

Open an **Administrator** PowerShell on the server. First find out what your adapter is actually called — do not guess:

```
Get-NetAdapter | Format-Table Name, InterfaceDescription, MacAddress, Status -AutoSize
Get-NetIPAddress -AddressFamily IPv4 | Format-Table InterfaceAlias, IPAddress, PrefixLength
```

> **The lesson from Ubuntu, applied to Windows**
>
> On Ubuntu the netplan file silently discarded your static IP because of one misspelled key. Windows has the same class of problem: an address applied incorrectly survives until the next reboot and then vanishes. The permanent methods are the `NetTCPIP` cmdlets or `netsh interface ip set address`. Verify with a reboot, exactly as you did before.

```
# 10.0.0.50 on the intnet adapter. Replace "Ethernet" with YOUR adapter name from above.
Set-NetIPInterface -InterfaceAlias "Ethernet" -Dhcp Disabled
New-NetIPAddress -InterfaceAlias "Ethernet" -IPAddress 10.0.0.50 -PrefixLength 24

# Equivalent, if you prefer netsh:
netsh interface ipv4 set address name="Ethernet" static 10.0.0.50 255.255.255.0
```

Verify — and verify again *after a reboot*:

```
Get-NetIPAddress -InterfaceAlias "Ethernet" -AddressFamily IPv4
ipconfig
```

Name it properly

```
Rename-Computer -NewName "WINSRV-01"; Restart-Computer
```

Never work with a server called `WIN-4F2K9D8Q7P3A`. Hostnames appear in every log line, every alert and every report.

P1.5 First-boot housekeeping

Task	Why it matters here
Set the timezone deliberately	Every alert timestamp is compared against your notes. A wrong timezone makes a whole investigation confusing.
Install updates, then a second reboot	You want a stable baseline before measuring anything. Most updates need two reboots.

Create a named admin account	Daily use of the built-in `Administrator` is itself a finding. Create `labadmin`, use it, then disable the built-in account (P4.1).
Install OpenSSH Server	Server 2022 does not include it, and you want SSH from Kali and the host. `Add-WindowsCapability -Online -Name OpenSSH.Server~~~~0.0.1.0` `Start-Service sshd; Set-Service sshd -StartupType Automatic` `New-NetFirewallRule -DisplayName "OpenSSH" -Direction Inbound -Protocol TCP -LocalPort 22 -Action Allow`
Check Defender is actually running	Part P4.7. Some evaluation images ship with it off.
Record your baseline	SCA pass rate and alert count *now*. You cannot show improvement without a before.

Your emergency command channel, on Windows too

WinServer has Guest Additions, so the out-of-band channel from Volume 1 works here — and it needs neither SSH nor networking. It is how you recover when the firewall drops your SSH port, which will happen at least once:

```
VBoxManage guestcontrol "WinServer" run --username .\labadmin --password <pw> `
  --exe "C:\Windows\System32\cmd.exe" -- /c "whoami"
```

P2 Enroll it in Wazuh

Once this works, everything from Part P5 onwards becomes possible: you can see what the server is doing instead of guessing at it.

P2.1 The enrolment flow, in order

1. Register → 2. Install agent → 3. Open firewall 1514 → 4. Verify Active → 5. Confirm alerts flow

> **Step 3 is where this almost always fails. Do not skip it.**
>
> Windows Firewall blocks inbound TCP by default. The Wazuh agent installs cleanly, starts, shows as running — and silently sends nothing. This is the most common Windows enrolment failure there is, and it looks exactly like a Wazuh problem.

P2.2 Register the agent

In the dashboard: **Agents → Add agent → Windows**. Fill in:

- **Name:** `winsrv-01` — matches the hostname you set in P1.4
- **IP:** `10.0.0.50/24`
- **Tags:** `os.windows,role.server,env.lab`

Copy the generated PowerShell command. Note the key is unique to this agent.

P2.3 Install the agent

On WinServer, in an Administrator PowerShell. Either use the command the dashboard generated, or do it in explicit steps so you can see what is happening:

```
# 1. download (Option A with NAT). Skip this line if you sideloaded.
Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.8-1.msi `
  -OutFile $env:TEMP\wazuh-agent.msi

# 2. install silently, pointing at the lab IP
$msi = "$env:TEMP\wazuh-agent.msi"
Start-Process msiexec.exe -ArgumentList @(
  "/i", $msi, "/quiet",
  "WAZUH_MANAGER=10.0.0.40",
  "WAZUH_AGENT_NAME=winsrv-01",
  "WAZUH_AGENT_GROUP=windows,servers"
) -Wait

# 3. start it and make it survive reboot
Set-Service -Name "Wazuh" -StartupType Automatic
Start-Service -Name "Wazuh"
Get-Service -Name "Wazuh"     # expect: Running
```

Where the agent keeps its configuration, mirroring Linux:

```
Get-Content "C:\Program Files (x86)\ossec-agent\ossec.conf" |
  Select-String "server|port|agent_name|labels"
Get-Content "C:\Program Files (x86)\ossec-agent\wazuh-agent.conf" | Select-Object -First 40
```

P2.4 Open the firewall — the step everyone misses

```
# Inbound rule for the agent listener
New-NetFirewallRule -DisplayName "Wazuh Agent 1514" `
  -Direction Inbound -Action Allow -Protocol TCP -LocalPort 1514 -Profile Any

# Optional: restrict RDP to the lab network only, never from anywhere
New-NetFirewallRule -DisplayName "RDP from lab net only" `
  -Direction Inbound -Action Allow -Protocol TCP -LocalPort 3389 `
  -RemoteAddress 10.0.0.0/24 -Profile Any

# Confirm both
Get-NetFirewallRule -DisplayName "Wazuh*" | Format-Table DisplayName, Enabled, Direction, Action -
AutoSize
Get-NetFirewallProfile  | Format-Table Name, Enabled, DefaultInboundAction -AutoSize
```

> **Why only inbound 1514, and nothing else**
>
> The agent *initiates* connections out to the manager on port 1514 and receives replies. Outbound is allowed by default, so that direction needs no rule. The inbound 1514 rule lets the manager accept a push connection and makes troubleshooting from the manager straightforward. Do **not** expose 55000 or 9200 — those are the manager API and the indexer, and opening them to a lab subnet is a real and extremely common mistake.

P2.5 Verify, properly

```
# 1. Is it listening?
netstat -ano | Select-String ":1514|:55000"

# 2. Is the service running and set to auto?
Get-Service Wazuh | Format-Table Name, Status, StartType -AutoSize

# 3. Read the agent's own log - this tells you exactly what it is doing
Get-Content "C:\Program Files (x86)\ossec-agent\wazuh-agent.log" -Tail 30

# 4. Can it reach the manager? (this is the real test)
Test-NetConnection -ComputerName 10.0.0.40 -Port 1514
```

If you see	It means
`TcpTestSucceeded : True`	Network path is fine. If alerts still do not flow, the problem is Wazuh-side.
`Timestamps in wazuh-agent.log advancing`	Agent is alive and working.
`Enrolled new agent with name: winsrv-01`	Registration succeeded.
`ERROR: (4101): Waiting for server response`	Registered, but the manager is not answering. Check `agent_control -l` on SIEM-Ubuntu.
`Could not connect to server`	Wrong manager IP, the firewall, or a routing problem on intnet.
`Key mismatch / invalid key`	The key in the local config does not match the manager. Reset with `manage_agents -R` and reinstall.

P2.6 Confirm in the dashboard

Refresh **Agents**. Status should go Pending → Active within about 30 seconds. Then verify real data, not just a heartbeat:

```
rule.level:>=0 AND agent.name:"winsrv-01"
```

On a fresh server you should immediately see events from the **System** channel (service starts, boot), **Application**, and — once auditing is on in Part P4.4 — **Security**. If you see only a heartbeat and nothing else, the agent is connected but has nothing interesting to report yet — which is exactly the state you want to be in before hardening.

P2.7 Add labels so filtering stays sane

```
# In the dashboard: Agents -> winsrv-01 -> Labels
# or via the API from SIEM-Ubuntu
curl -sk -X POST "https://localhost:55000/agents/002/labels" `
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' `
  -d '{"labels":["env.lab","os.windows","os.server2022"]}'
```

Within a few weeks you will have many agents and no memory of what any of them are. Labels are how you answer “show me every domain controller” without remembering names.

P3 Audit it from Kali (read-only)

Before you change anything, know what you are dealing with. This whole part is observation only — nothing here modifies the server. It does generate log events, which is useful: you are building the alert baseline from Part P5 as a side effect.

P3.1 Reachability from Kali

```
ip -br a                                # you are 10.0.0.30
ping -c3 10.0.0.50
ip neigh | grep 10.0.0.50               # confirm the MAC matches WinServer
```

Expect `08:00:27:f7:2c:54` — the MAC from the VM config. A different MAC means you are looking at a different machine, or an address conflict has crept back in.

P3.2 Service and version audit

```
# Fast: the ports a Windows Server normally has open
sudo nmap -Pn -p 53,88,135,139,389,445,464,636,3268,3269,3389,5985,9389 \
     --open 10.0.0.50

# Full picture: every TCP port, version detection, default scripts
sudo nmap -Pn -sV -sC -p- --open 10.0.0.50 -oN ~/nmap-winsrv-full.txt

# Just the standard set, quick
sudo nmap -Pn -sV -F 10.0.0.50
```

Port	Service	Does the lab need it?
53	DNS	Only if you promote this to a domain controller (P9). Remove otherwise.
88	Kerberos	Domain controller only.
135	RPC endpoint mapper	Used by many Windows services. Do not disable — breaking it breaks management tools.
139	NetBIOS session	**Should be blocked.** Legacy and unnecessary on a modern network.
389	LDAP	Domain controller only.
445	SMB	Required, if only for administration. This is the one that matters most to harden.
464	Kerberos password change	Domain controller only.
3389	RDP	Only if you need the GUI. Consider removing and using SSH instead.
5985	WinRM	Only if you manage it remotely with PowerShell.
9389	AD DS	Domain controller only.

> **Read the scan output as a configuration review, not a score**
>
> An open port on a lab server is not automatically a problem — you need it for something. The question is always "does it need to be reachable, from where, and is it configured safely?" A default Windows Server answering on 445 to the whole lab with SMBv1 enabled and no signing required is a finding. A 445 open only to 10.0.0.30 with signing required is fine.

P3.3 SMB configuration audit

```
sudo nmap -Pn -p 445 --script \
  smb-os-discovery,smb-protocols,smb2-security-mode,smb2-time,smb-enum-shares \
  10.0.0.50
```

Script	What a good result looks like
`smb-os-discovery`	Confirms the build. Compare with 20348.587 in your VM config.
`smb-protocols`	**SMBv1 should be absent.** If it reports `SMBv1 enabled`, that is finding #1.
`smb2-security-mode`	**SMB signing should be required or at least enabled.** "disabled" is finding #2.
`smb-enum-shares`	Compare against what you expect. Any `ADMIN$` / `C$` reachable from the lab is worth noting.

```
# Enumerate shares with a null session (read-only)
smbclient -L //10.0.0.50 -N
smbmap -H 10.0.0.50

# OS and domain details, no credentials needed
rpcclient -U "" -N -c "srvinfo" 10.0.0.50
```

> **Record which of these succeeded**
>
> Whether anonymous share listing works is itself a finding. On a hardened server it should fail. Note it now, then note it again in Part P5 after you have locked it down — the difference is your evidence.

P3.4 Password policy, read from both sides

From the server itself (authoritative)

```
net accounts
Get-LocalUser | Format-Table Name, Enabled, PasswordExpires, PasswordLastSet -AutoSize
Get-LocalGroupMember -Group "Administrators" | Format-Table Name, PrincipalSource -AutoSize
```

From Kali, over the network

```
# confirms the policy from outside - useful for your report
netexec smb 10.0.0.50 --pass-pol
```

This reads the policy without needing any credentials. Note the result and compare it to `net accounts` on the server. If they differ, the network view is the one an outsider would have.

P3.5 Known-vulnerability check

```
sudo nmap -Pn -p 445 --script vuln 10.0.0.50 -oN ~/nmap-vuln-winsrv.txt
less ~/nmap-vuln-winsrv.txt
```

This queries public vulnerability data for the services it detects. Treat the output as a prompt to investigate, not as proof. A CVE listed means "this version is associated with a known issue" — you still need to confirm it applies, then fix it in Part P4 or document why it does not.

P3.6 Write down your baseline

Before touching anything, record this

Item	Observed value
Open ports	
SMBv1 supported?	
SMB signing	
SMB encryption	
Anonymous share listing	
Minimum password length	
Password complexity	
Account lockout threshold	
Guest account enabled?	
Firewall profiles enabled	
SCA pass rate (System Audit → CIS)	
Wazuh alert count from winsrv-01	

This table is your "before". Part P4 changes it and Part P5 proves you can see the change.

P4 Harden it — the main event

Ordered roughly by impact on your SIEM. Sections P4.1 to P4.3 are good practice on any server; **P4.4 is the one your Wazuh lab cannot work without.**

> **Take a snapshot before starting this part**
>
> Several of these changes are not easily reversible, particularly Credential Guard and LSA protection. Power off and snapshot as `winserver-audited-30sep`. Recovering a server you cannot log into is a genuinely bad afternoon, and this makes it impossible.

P4.1 Accounts and passwords

```
# --- current state ---
net accounts
Get-LocalUser | Format-Table Name, Enabled -AutoSize

# --- password policy: min 14 chars, max age 60 days ---
net accounts /minpwlen:14 /minpwage:1 /maxpwage:60 /lengthenable:yes

# --- account lockout: 5 failures, 15 minutes ---
net accounts /lockoutthreshold:5 /lockoutduration:15 /lockoutobservation:30

# --- disable the Guest account ---
net user guest /active:no

# --- who is actually an admin? ---
Get-LocalGroupMember -Group "Administrators" | Format-Table Name, PrincipalSource -AutoSize
net localgroup Administrators
```

Control	Why
Minimum length ≥ 14	The single most effective password control. Length beats complexity rules — people reuse complex passwords and guess long ones.
Maximum age 30–60 days	Forces rotation. There is real disagreement in the industry about whether this helps; the CIS benchmark requires it, so the benchmark wants it.
Lockout at 5 / 15 minutes	Turns online guessing into a non-viable strategy. **Know the trade-off:** it can also be used to lock out a legitimate user. That is a known denial-of-service consideration you should be able to discuss in an interview.
Guest disabled	Guest has almost no privileges and is a target because it is enabled by default.

> **Use a named admin account, not `Administrator`**
>
> Create `labadmin`, add it to Administrators, use it for everything, then disable the built-in account. This is a standard CIS recommendation: a guessable, always-enabled, universally-known account name is a gift. Be certain your `labadmin` credentials work before disabling `Administrator`.

```
New-LocalUser -Name "labadmin" -Password (Read-Host -AsSecureString) `
  -Description "Lab admin account"
Add-LocalGroupMember -Group "Administrators" -Member "labadmin"
# test the new login FIRST, in a second session, then:
Disable-LocalUser -Name "Administrator"
```

Remove stale accounts

```
Get-LocalUser | Where-Object { $_.LastLogon -lt (Get-Date).AddMonths(-6) }
Get-LocalUser | Where-Object { $_.PasswordLastSet -eq $null }   # never-set password
```

P4.2 SMB hardening — the highest-value technical change

Check the current configuration

```
Get-SmbServerConfiguration | Format-List EnableSMB1Protocol, EnableSMB2Protocol, `
  EncryptData, RequireSecuritySignature, EnableSecuritySignature, `
  EnableAuthNullUser, EnableInsecureGuestLogons
```

Apply

```
# 1. turn off SMBv1
Get-WindowsOptionalFeature -Online -FeatureName SMB1Protocol
Disable-WindowsOptionalFeature -Online -FeatureName SMB1Protocol -NoRestart

# 2. require SMB signing on all server-side connections
Set-SmbServerConfiguration -RequireSecuritySignature $true `
                           -EnableSecuritySignature  $true -Force

# 3. require SMB encryption for all shares
Set-SmbServerConfiguration -EncryptData $true -Force

# 4. deny null / anonymous sessions
Set-SmbServerConfiguration -EnableAuthNullUser $false -Force
Set-SmbServerConfiguration -EnableInsecureGuestLogons $false -Force
```

Verify — the same command you used to read it should now report the new values:

```
Get-SmbServerConfiguration | Format-List EnableSMB1Protocol, RequireSecuritySignature, EncryptData
```

> **Test administration after this, not before**
>
> Requiring SMB signing or encryption will break management tools that connect to `\\WINSRV-01\C$` from a machine that does not support the new requirement. If you administer from your Windows host, verify a remote administration path works *before* you apply it. SSH is unaffected, which is another good reason to have installed it in P1.5.

Restrict anonymous access in the registry

```
$lsa = "HKLM:\SYSTEM\CurrentControlSet\Control\Lsa"
Set-ItemProperty $lsa -Name RestrictAnonymous       -Value 1
Set-ItemProperty $lsa -Name RestrictAnonymousSAM    -Value 1
Set-ItemProperty $lsa -Name restrictnullsessions    -Value 1
Set-ItemProperty $lsa -Name EveryoneIncludesAnonymous -Value 0
Set-ItemProperty "HKLM:\SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters" `
  -Name RestrictNullSessAccess -Value 1

# confirm
Get-ItemProperty $lsa | Select-Object RestrictAnonymous, RestrictAnonymousSAM, restrictnullsessions
```

P4.3 Protecting credentials on the server itself

LSA protection (RunAsPPL)

Without this, software running on the server can read process memory and extract credentials. With it, the Local Security Authority runs as a protected process and that technique fails.

```
$lsa = "HKLM:\SYSTEM\CurrentControlSet\Control\Lsa"
Set-ItemProperty $lsa -Name RunAsPPL     -Value 1
Set-ItemProperty $lsa -Name RunAsPPLBoot -Value 1
Restart-Computer
```

Credential Guard

Runs the LSA in a container isolated from the rest of the OS, using virtualisation-based security. Stronger than LSA protection, and the two work well together.

```
Enable-WindowsOptionalFeature -Online -FeatureName CredentialGuard -All
```

Verify with Microsoft's own tool — it prints a table of which protections are active:

```
C:\Windows\System32\lsaasg.exe /query
```

Expect `Credential Guard: Enabled` and `LSA protection (RunAsPPL): Enabled`. A reboot is generally needed for the full effect.

> **Why these two matter, in one sentence**
>
> Both exist to stop a process that *already has administrator* from reading plaintext credentials out of memory. That is the most common technique in real Windows intrusions — and it is why "the attacker is already an admin" is not the end of the story.

P4.4 Audit policy — THE most important section for your SIEM

> **Read this part twice**
>
> Without this, Wazuh will show you almost nothing about Windows. With it, you can detect logons, privilege changes, file access, process creation and log tampering — which is the entire difference between a Windows machine in your SIEM and a black box in it.

Back up the current policy first

```
auditpol /backup /file:C:\Windows\Temp\audit-backup.csv
auditpol /get /category:* | Out-File C:\Windows\Temp\audit-current.txt
auditpol /get /subcategory:*
```

> **This one line makes the whole section reversible**
>
> `auditpol /restore /file:C:\Windows\Temp\audit-backup.csv` puts everything back exactly as it was. Do the backup before anything else in this section, no exceptions.

Turn on the auditing Wazuh needs

```
# --- authentication: the foundation ---
auditpol /set /subcategory:"Logon" /success:enable /failure:enable        # 4624 / 4625
auditpol /set /subcategory:"Logoff" /success:enable                       # 4634 / 4635
auditpol /set /subcategory:"Special Logon" /success:enable                 # 4672  privileged logon
auditpol /set /subcategory:"Logon (Service Package)" /success:enable     # 4648  explicit creds
auditpol /set /subcategory:"Credential Validation" /success:enable /failure:enable # 4776 NTLM
auditpol /set /subcategory:"Account Lockout" /failure:enable               # 4740  lockout fired

# --- privilege and account changes: the high-value ones ---
auditpol /set /subcategory:"Security Group Management" /success:enable    # 4728 / 4732 / 4756
auditpol /set /subcategory:"User Account Management" /success:enable      # 4720 / 4722 / 4724 /
4726
auditpol /set /subcategory:"Audit Policy Change" /success:enable          # 4719  auditing changed!

# --- file and object access ---
auditpol /set /subcategory:"Detailed File Share" /success:enable          # 5140
auditpol /set /subcategory:"Other Object Access Events" /success:enable # 5145  noisy - see note

# --- process creation ---
auditpol /set /subcategory:"Process Creation" /success:enable           # 4688  HIGH VOLUME

# --- system state ---
auditpol /set /subcategory:"Security State Change" /success:enable      # 4616
auditpol /set /subcategory:"Other Account Logon Events" /success:enable # 4647
auditpol /set /subcategory:"Security Group Membership" /success:enable # 4727 / 4729
```

Include the command line in process creation events

Without this, event 4688 tells you *what* ran but not *with what arguments* — which is most of the useful information.

```
Set-ItemProperty "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" `
  -Name AuditProcessCreationIncludeCmdLine_Enabled -Value 1

# Windows 10 / Server 2016 and later: also required for the full command line
Set-ItemProperty "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Sys\CmdLine" `
  -Name AuditAdditionalProc -Value 1
```

Volume warning — read before enabling process auditing

4688 with command lines generates thousands of events per hour on a busy machine. On your 4 GB lab server it is manageable, but it will be the dominant source of alerts, it will enlarge your indexer indices, and it will bury everything else unless you filter deliberately. Enable it, look at the volume, and if it is overwhelming, disable it and rely on Sysmon instead (Part P6), which gives comparable detail with less duplication. Document whichever you choose and why — that judgement is the actual skill.

Verify

```
auditpol /get /subcategory:"Logon"
auditpol /get /subcategory:"Security Group Management"
auditpol /get /category:"Detailed Tracking"
```

How to prove auditing works, immediately, with nothing but your own actions

```
wevtutil qe Security /c:10 /rd:true /f:text | Select-String "4624|4672|4732"
```

Do something privileged from a second session, then run that. That is the fastest possible audit of whether your audit policy is doing its job.

P4.5 Windows Firewall

```
# Current state - all three profiles; note which are even enabled
Get-NetFirewallProfile | Format-Table Name, Enabled, DefaultInboundAction, DefaultOutboundAction -
AutoSize

# Harden: all profiles on, deny inbound by default
Set-NetFirewallProfile -Profile Domain,Public,Private `
  -Enabled True -DefaultInboundAction Block -DefaultOutboundAction Allow

# Re-verify, and re-verify your Wazuh rule still exists
Get-NetFirewallProfile | Format-Table Name, Enabled, DefaultInboundAction -AutoSize
Get-NetFirewallRule -DisplayName "Wazuh*" | Format-Table DisplayName, Enabled -AutoSize
```

Order of operations matters here

`Set-NetFirewallProfile` can disturb existing rules. Create your inbound allows *first* (P2.4), then set the profiles. Reverse the order and you can lock yourself out of RDP — and with the built-in Administrator disabled, recovery becomes genuinely painful. Keep the VirtualBox console open.

Review what is exposed

```
Get-NetFirewallRule -Direction Inbound -Enabled True -Action Allow |
  Format-Table DisplayName, Profile -AutoSize

Get-NetFirewallRule -DisplayName "Remote*" | Format-List DisplayName, Enabled, Profile
```

P4.6 RDP

Two reasonable positions. Pick one and be able to justify it.

Option 1 — remove RDP entirely, manage over SSH

```
Set-ItemProperty "HKLM:\System\CurrentControlSet\Control\Terminal Server" `
  -Name fDenyTSConnections -Value 1
Restart-Service -Name TermService -Force
Disable-NetFirewallRule -DisplayName "Remote*Desktop*"
```

Option 2 — keep RDP, but restrict it hard

```
# enable, with Network Level Authentication (the important part)
Set-ItemProperty "HKLM:\System\CurrentControlSet\Control\Terminal Server" `
  -Name fDenyTSConnections -Value 0
Set-ItemProperty "HKLM:\System\CurrentControlSet\Control\Terminal Server\WinStations\RDP-Tcp" `
  -Name UserAuthentication -Value 1

# no drive or clipboard redirection
Set-ItemProperty "HKLM:\System\CurrentControlSet\Control\Terminal Server\WinStations\RDP-Tcp" `
  -Name fDisableCdm -Value 1
```

Setting	Why
NLA (UserAuthentication=1)	Requires authentication *before* a session is created. Without it, RDP is reachable and consuming resources before any login is attempted — which is exactly how it gets abused.
fDisableCdm=1	Stops the client mapping local drives and clipboard into the server, closing an easy path for moving data in or out.
Firewall scoped to 10.0.0.0/24	RDP should not be reachable from an entire network. Scope it to the management segment.

P4.7 Defender

```
Get-MpComputerStatus | Format-List AMServiceEnabled, AntivirusEnabled, `
  RealTimeProtectionEnabled, BehaviorMonitorEnabled, IoavProtectionEnabled, `
  AntivirusSignatureLastUpdated

Get-MpPreference | Format-List DisableRealtimeMonitoring, ExclusionPath
```

```
# ensure real-time protection is on
Set-MpPreference -DisableRealtimeMonitoring $false
Set-MpPreference -DisableBehaviorMonitoring $false

# update signatures
Update-MpSignature
```

> **Check exclusions on purpose**
>
> `Get-MpPreference | Select ExclusionPath` deserves a proper look. Antivirus exclusions are a classic attacker technique — adding one is a single command and it disables your endpoint protection invisibly. It is also why "Defender was disabled" is an incident, not a configuration note. Read the list and note anything you cannot explain.

P4.8 PowerShell logging

PowerShell is the most heavily abused tool on Windows and the one you are least able to detect without planning for it. Log it before you need it.

```
$ps = "HKLM:\Software\Policies\Microsoft\Windows\PowerShell"
New-Item -Path "$ps\ScriptBlockLogging" -Force | Out-Null
Set-ItemProperty -Path "$ps\ScriptBlockLogging" -Name EnableScriptBlockLogging -Value 1
Set-ItemProperty -Path "$ps\ScriptBlockLogging" -Name EnableModuleLogging -Value 1

New-Item -Path "$ps\Transcription\FilePath" -Force | Out-Null
Set-ItemProperty -Path "$ps\Transcription\FilePath" `
  -Name FilePath -Value "C:\ProgramData\WazuhLOGS\PowerShellTranscripts"
New-Item -ItemType Directory -Path "C:\ProgramData\WazuhLOGS" -Force | Out-Null
```

```
# 4104 = ScriptBlockLogging, 4103 = module logging
wevtutil qe "Microsoft-Windows-PowerShell/Operational" /c:3 /rd:true /f:text |
  Select-String "4104|ScriptBlock"
```

> **The proper way is Group Policy, and it is worth learning**
>
> The registry writes above are what you use standalone. With Group Policy (`gpedit.msc` → Administrative Templates → Windows Components → Microsoft Windows → PowerShell), the same settings are named checkboxes, which is far less error-prone. Learn GPO properly when you promote this to a domain controller — it is the central tool of Windows administration and you will use it constantly.

P4.9 Re-audit and compare

Repeat Part P3 exactly. Same commands, same order, so the comparison is honest.

```
sudo nmap -Pn -p 445 --script smb-protocols,smb2-security-mode,smb-enum-shares 10.0.0.50
smbclient -L //10.0.0.50 -N
netexec smb 10.0.0.50 --pass-pol
```

Build the before / after table

Check	Before	After
SMBv1		absent
SMB signing		required
SMB encryption		required
Anonymous share list		denied
Min password length		14
Lockout threshold		5 / 15 min
Guest account		disabled
Null sessions (registry)		restricted
LSA protection		enabled
Credential Guard		enabled
Audit: group management		success enabled
Audit: process creation		enabled + cmdline
SCA pass rate		

This table, with the actual outputs pasted in, is the centrepiece of your portfolio. It demonstrates you understand *why* each control exists, which is what an interviewer is actually testing.

P5 Validate your detection

You now have a hardened server enrolled in a SIEM. But does the SIEM *actually see* things? This part proves it, with evidence, by generating controlled activity on your own host.

> **What makes this legitimate, concretely**
>
> Every action below is performed by you, on a machine you own, using your own credentials or a throwaway account you created for this purpose. You are not attacking anything — you are *generating log events so your sensor has something to detect*. Nothing here tries to gain access to anything, because you already have administrator. That is the whole difference.

P5.1 The method

Choose a technique → Note the event ID → Perform it → Confirm Wazuh alerted → Record the gap, or fix it

Every test ends with one of two honest outcomes, and **both are valuable**: "detected, here is the alert" or "not detected, here is the rule I need to write." A SOC that has only ever seen detections on a dashboard it configured itself does not know its gaps. Finding and closing them is the job.

P5.2 Test 1 — Service and port discovery

Why this matters: service enumeration is the standard first step of reconnaissance. If your SIEM cannot see it, it cannot see the next step either.

```
# from Kali
sudo nmap -Pn -sV -p- --open 10.0.0.50 --max-rate 500
```

Where to look in Wazuh:

```
agent.name:"winsrv-01" AND (rule.groups:portscan OR rule.description:*portscan*)
agent.name:"winsrv-01" AND location:"win_eventchannel"
```

Write down: did it alert? If not, that is a gap you close in Part P7.

P5.3 Test 2 — SMB share enumeration

```
# from Kali - read-only enumeration
smbclient -L //10.0.0.50 -N
smbmap -H 10.0.0.50
```

If your null-session restriction from P4.2 is working, these should now **fail**:

```
tree connect failed: NT_STATUS_ACCESS_DENIED
```

That failure is a good result in itself, and it should still have produced an audit event on the server. Check:

```
data.win.system.eventID:5140 OR data.win.system.eventID:5145
agent.name:"winsrv-01" AND data.win.system.eventID:5145
```

> **If you see nothing at all**
>
> That is a real and common finding: the "Other Object Access Events" audit subcategory from P4.4 is what generates 5145. If it is not enabled, Wazuh is blind to file access. Go back and enable it. This is exactly the kind of gap this part exists to find.

P5.4 Test 3 — Failed authentication, and lockout

This is the test most people want to do first, so let us be precise about what it is.

1 Create a throwaway test account on the server

```
# on WinServer, as labadmin
$pw = ConvertTo-SecureString "A7#kQ2mZ9xR4tB1vC6nH8sD3fG5jL0w" -AsPlainText -Force
New-LocalUser -Name "labtestuser" -Password $pw -PasswordNeverExpires $true `
              -Description "Throwaway account used only for detection testing"
Add-LocalGroupMember -Group "Users" -Member "labtestuser"
```

You will never need to type that password. You will deliberately send the **wrong** value, a fixed number of times, to a username you own. That produces log events. It is not an attack, there is no wordlist, and there is no discovery involved.

2 Generate the failures from Kali, deliberately

```
# ONE known-wrong password, a known number of times, to YOUR OWN account
for i in $(seq 1 8); do
  smbclient //10.0.0.50/IPC$ -U "labtestuser%definitely-not-the-password" -c "quit"
>/dev/null 2>&1
  sleep 1
done
```

> **Why this is fine, stated plainly**
>
> You own both endpoints. The account is one you just created. The password you send is one wrong value you typed on purpose, not a list. The purpose is to make your SIEM emit "failed logon" events so you can confirm it is ingesting and decoding them. Nothing here discovers, targets, or gains access to anything.

3 Find the events in Wazuh

```
rule.groups:authentication_failed AND agent.name:"winsrv-01"
data.win.system.eventID:4625 AND agent.name:"winsrv-01"
```

You should see roughly 8 failed logons, then a lockout event once you pass your threshold of 5.

4 Find the lockout — event 4740

```
data.win.system.eventID:4740
```

This is the one that matters most. A burst of failed logons is noise; an account actually being locked out is a signal. Check whether your frequency rule from Part P7 fires, and whether Wazuh's stock rules correlate the eight failures into one meaningful alert rather than eight separate ones. **That judgement — noise versus signal — is the actual detection skill.**

5 Clean up

```
# on WinServer
Unlock-LocalUser -Name "labtestuser"
Remove-LocalUser -Name "labtestuser"
```

Leaving a test account with a known password on a server is precisely the thing you would criticise in someone else's configuration. Do not leave it.

P5.5 Test 4 — Successful logon from a new source

```
# from Kali, with the REAL password, using your own account
smbclient //10.0.0.50/IPC$ -U "labadmin%YourRealPassword" -c "quit"
```

```
# then in Wazuh
data.win.system.eventID:4624 AND agent.name:"winsrv-01"
data.win.system.eventID:4624 AND data.win.eventdata.LogonType:3   # type 3 = network
```

Logon type	Meaning
2	Interactive — someone at the console or via RDP
3	Network — accessing a share or a web application, or via `smbclient`
4	Batch — scheduled task
5	Service — a Windows service starting
7	Unlock — unlocking a locked console
10	RemoteInteractive — RDP
11	CachedInteractive — offline logon to a cached account

Also look for **4648** (explicit credential use). A network logon with explicit credentials is how `runas` and remote-execution tooling work, and it is a classic way to conceal which account is really being used.

P5.6 Test 5 — Privilege escalation indicator

Among the highest-value detections in Windows, because it is what usually follows a successful compromise.

```
# on WinServer, as labadmin - add the throwaway test user to Administrators, then remove it
Add-LocalGroupMember -Group "Administrators" -Member "labtestuser"
# ... observe in Wazuh ...
Remove-LocalGroupMember -Group "Administrators" -Member "labtestuser"
```

```
# in Wazuh
data.win.system.eventID:4732 AND agent.name:"winsrv-01"
data.win.system.eventID:4672 AND agent.name:"winsrv-01"   # special privileges
```

> **Why 4732 is the one to alert on**
>
> A member being added to the **Administrators** group is a far stronger signal than almost any other single Windows event. In a real incident it is often the moment an intrusion first becomes visible. It is the first rule you should write in Part P7.

P5.7 Test 6 — Persistence via scheduled task

```
# on WinServer - create, observe, then delete
schtasks /create /tn "LabDetectionTest" /tr "C:\Windows\System32\whoami.exe" `
         /sc ondemand /ru labadmin /f
schtasks /query /tn "LabDetectionTest" /v /fo list
schtasks /delete /tn "LabDetectionTest" /f
```

```
# in Wazuh - 4698 lives in the Security channel
data.win.system.eventID:4698 AND agent.name:"winsrv-01"
data.win.system.eventID:4699 AND agent.name:"winsrv-01"   # task deleted
```

Scheduled tasks are among the most common persistence mechanisms. A rule that alerts on task creation in directories you control is high value and low noise.

P5.8 Test 7 — FIM: did anyone change a file?

Configure Windows FIM for the directories that matter, then touch a file and watch it appear. Edit `C:\Program Files (x86)\ossec-agent\ossec.conf`:

```
<syscheck>
  <frequency>7200</frequency>
  <directories>
    <directory>C:\ProgramData\WazuhLOGS</directory>
    <directory>C:\inetpub</directory>
    <directory>C:\Windows\Temp</directory>
    <directory>C:\Windows\System32\drivers\etc</directory>
  </directories>
  <nodiff>
    <path>C:\ProgramData\WazuhLOGS\</path>   # transcripts - too noisy to hash
  </nodiff>
  <registry_ignore>
    <arch>64</arch>
    <path>Microsoft\Windows\CurrentVersion\Explorer</path>
  </registry_ignore>
</syscheck>
```

```
Restart-Service Wazuh
Add-Content "C:\Windows\Temp\lab-fim-test.txt" "harmless test content"
```

```
# then in Wazuh
syscheck.file:C:\Windows\Temp\lab-fim-test.txt
location:syscheck AND agent.name:"winsrv-01"
```

P5.9 Test 8 — Log tampering

One of the highest-value detections in all of security: if someone clears the audit log, they know they were noticed and are covering tracks.

> **Do this test with extreme care, and only once**
>
> Clearing the Security log destroys the history you need. Before doing it: (1) take a VM snapshot, (2) export the current log, (3) be certain you have SSH or the console available. Better: verify the *rule* first, and only run the live test if you really want the proof.
>
> ```
> # 1. back it up BEFORE anything else
> wevtutil epl Security C:\Windows\Temp\Security-backup.evtx
> # 2. confirm you can still get in another way
> # 3. only then, if you choose to proceed:
> #    Clear-EventLog -LogName Security
> ```

When you do run it, the resulting event is **1102** "The audit log was cleared", and it is generated *as part of* the clear:

```
data.win.system.eventID:1102
rule.description:*log*cleared*
rule.groups:log_cleared
```

This detection works even if the attacker clears everything else, because Wazuh is already streaming the event to the indexer on another machine. That architectural property — centralised, remote, tamper-resistant logging — is the strongest argument for a SIEM, and this test is the most convincing demonstration of it you can run.

P5.10 Test 9 — Service installation

```
# on WinServer - 7045 lives in the System channel, not Security
sc.exe create LabTestService binPath= "C:\Windows\System32\cmd.exe /c whoami"
sc.exe delete LabTestService
```

```
rule.description:"New service installed"
data.win.system.eventID:7045
agent.name:"winsrv-01" AND location:"win_eventchannel"
```

P5.11 The validation matrix

Fill this in completely. It is the single most useful page in your whole project.

#	Test	Event ID	Expected source	Detected?	Notes / rule needed
1	Port scan	`5156` / `5157`	Security		
2	Share enumeration	`5140` / `5145`	Security		
3a	Failed logon	`4625`	Security		
3b	Account lockout	`4740`	Security		
4	Successful network logon	`4624`	Security		
4b	Explicit credentials	`4648`	Security		
5a	Added to Administrators	`4732`	Security		
5b	Special privileges	`4672`	Security		
6	Scheduled task created	`4698`	Security		
7	File change (FIM)	`syscheck`	syscheck		
8	Audit log cleared	`1102`	Security		
9	Service installed	`7045`	System		
10	User account created	`4720`	Security		
11	Audit policy changed	`4719`	Security		
12	Process created (cmdline)	`4688`	Security		
13	PowerShell script block	`4104`	PS Operational		

P5.12 What to do with the gaps

Every blank cell in “Detected?” is a finding. For each one, decide:

Reason for the gap	Fix
Audit policy not enabled	Part P4.4. Enable the subcategory.
Log channel not collected	Part P6. Add it to the agent config.
Event collected but no rule fires	Part P7. Write a local rule.
Event collected but rule too low	Part P7. Raise the level.
Deliberately not collected	Write down *why*. A documented decision is a finding too.

P6 Windows event coverage and Sysmon

Wazuh's Windows agent does not collect every log by default. This section closes the coverage gaps you found in Part P5.

P6.1 What is collected by default

Channel	Contains
`Application`	Application errors and warnings
`Security`	**The important one.** Logons, privileges, audit events. Nearly empty until P4.4 is done.
`System`	Service start/stop, driver loads, service installations (7045)
`Windows PowerShell`	PowerShell engine events, if configured

P6.2 Adding channels

Edit `C:\Program Files (x86)\ossec-agent\ossec.conf`. The block you want is `localfile`, which maps a log channel to a decoder:

```
<ossec_config>
  <localfile>
    <log_format>eventchannel</log_format>
    <location>Microsoft-Windows-PowerShell/Operational</location>
  </localfile>

  <localfile>
    <log_format>eventchannel</log_format>
    <location>Microsoft-Windows-TaskScheduler/Operational</location>
  </localfile>

  <localfile>
    <log_format>eventchannel</log_format>
    <location>Microsoft-Windows-Windows Firewall With Advanced Security/Firewall</location>
  </localfile>

  <localfile>
    <log_format>eventchannel</log_format>
    <location>Microsoft-Windows-Sysmon/Operational</location>
  </localfile>
</ossec_config>
```

```
Restart-Service Wazuh
Get-Content "C:\Program Files (x86)\ossec-agent\wazuh-agent.log" -Tail 20
```

Verify the channel exists before adding it — a typo is silently ignored:

```
Get-WinEvent -ListLog * | Where-Object { $_.LogName -like "*PowerShell*" } |
  Select-Object LogName, IsEnabled
wevtutil gli "Microsoft-Windows-Sysmon/Operational"
```

> **log_format matters more than you would think**
>
> `eventchannel` is for modern Windows Event Log channels and produces structured, decoded JSON. `full` is for text log files. Using `full` against an event channel gives you unparseable junk; using `eventchannel` against a text file does nothing at all. Most "my Windows agent sees nothing" problems are exactly this.

P6.3 Sysmon — the single best telemetry upgrade

Sysmon is part of Sysinternals (Microsoft). It writes one consolidated, high-quality log of process creation, network connections, file creation and registry changes — precisely the things that matter for detection. **Wazuh decodes Sysmon natively**, so no custom decoder is needed.

Install

```
# download Sysmon from the official Sysinternals site, then:
Expand-Archive -Path .\Sysmon.zip -DestinationPath C:\Sysmon
cd C:\Sysmon
.\Sysmon64.exe -i          # install with default config
.\Sysmon64.exe -c          # check the active config
```

Configure

The default config is reasonable, but a tuned config is much better. Download the vetted configuration published with Sysinternals and apply it:

```
.\Sysmon64.exe -c .\SysmonConfig.xml
Restart-Service Wazuh
```

Sysmon config is XML with two kinds of rule — *include* (capture these) and *exclude* (ignore these, to control volume). Anything neither included nor excluded is **not** captured. A minimal config covering the detection-relevant events:

```
<Sysmon schemaversion="4.90">
  <EventFiltering>
    <!-- Event ID 1: process creation, with full command line -->
    <ProcessCreate onmatch="include">
      <CommandLine condition="is any"/>
      <OriginalFileName condition="is any"/>
    </ProcessCreate>

    <!-- Event ID 3: outbound network connections -->
    <NetworkConnect onmatch="include">
      <DestinationPort condition="is any"/>
    </NetworkConnect>

    <!-- Event ID 11: file creation -->
    <FileCreate onmatch="include">
      <TargetFilename condition="is any"/>
    </FileCreate>

    <!-- Events 12/13/14: registry create / set / rename -->
    <RegistryEvent onmatch="include">
      <TargetObject condition="contains">CurrentVersion\Run</TargetObject>
      <TargetObject condition="contains">CurrentVersion\RunOnce</TargetObject>
      <TargetObject condition="contains">Services\</TargetObject>
      <TargetObject condition="contains">Windows Defender</TargetObject>
    </RegistryEvent>

    <!-- Event ID 22: DNS queries -->
    <DnsQuery onmatch="exclude">
      <QueryName condition="is any">localhost</QueryName>
    </DnsQuery>
  </EventFiltering>
</Sysmon>
```

> **Sysmon is very easy to flood with volume**
>
> Event IDs 1 and 11 produce enormous numbers of records on any active machine. Start with narrow *include* conditions, watch the volume for an hour, and widen only if you need to. Remember Sysmon events arrive in Wazuh as `location:win_eventchannel`, so if your alert count explodes after enabling it, that is why — and it is a good problem to have, because it means you have process-level visibility.

Verify Sysmon is reporting

```
# on WinServer
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 5 |
  Format-List Id, TimeCreated, Message
```

```
# then in Wazuh
data.win.system.eventID:1 AND agent.name:"winsrv-01"
data.win.system.eventID:3 AND agent.name:"winsrv-01"
location:win_eventchannel AND rule.level:>=9
```

P6.4 A better way to work than raw event IDs

Raw IDs are fine for testing, but for dashboards you want Wazuh's own rule *groups*, because they are stable across Wazuh versions and translate into plain English:

Group	Covers
`authentication_failed`	Failed logons
`authentication_success`	Successful logons
`account_changed`	Account and group modifications
`win_authentication`	Broad authentication family
`log_cleared`	Log tampering
`windows_privilege_change`	Your 4732 rule from P7.2
`windows_persistence`	Your task/service rules
`windows_correlation`	Your correlation rule

P7 Detection rules for Windows

Part P5 told you what the gaps are. This part closes them. The mechanics are in Volume 1 Part 13; here is what to write for Windows specifically.

P7.1 Find the field names first

Never guess a field name. Paste a real Windows event into the tester:

```
# on SIEM-Ubuntu
sudo /var/ossec/bin/wazuh-logtest
```

Better still, have the agent print its own decoded fields directly against the real Windows machine:

```
# on WinServer
& "C:\Program Files (x86)\ossec-agent\wazuh-agent.exe" -t
# paste a Security log line, press Enter
# it prints the rule that matched AND the decoded fields
```

> **The field names you will use constantly on Windows**
>
> `data.win.system.eventID` — the numeric event ID. And `data.win.eventdata.*` — the individual event fields, where the real detail lives (TargetUserName, SubjectUserName, IpAddress, LogonType). Spell them exactly as the tester prints them. Some Wazuh versions deliver the audit field as `data.win_system:eventID`, so trust the tester output over any documentation you read.

P7.2 The rules worth writing

Save into `/var/ossec/etc/local_rules.xml` on SIEM-Ubuntu. Validate and deploy:

```
sudo /var/ossec/bin/wazuh-logtest -t
sudo systemctl restart wazuh-manager
sudo /var/ossec/bin/agent_control -R 002   # refresh agent config
```

```
<group name="windows,lab_detection,">

  <!-- 4732: member added to a privileged group -->
  <rule id="100300" level="12">
    <if_sid>4732</if_sid>
    <description>Member $(MemberName) added to group $(TargetUserName) on $(SystemName)
</description>
    <mitre><id>T1098</id></mitre>
    <group>windows_privilege_change,critical_change,</group>
  </rule>

  <!-- 1102: audit log cleared. Serious, and rarely benign. -->
  <rule id="100301" level="14">
    <if_sid>1102</if_sid>
    <description>Audit log cleared on $(SystemName) by $(SubjectUserName)</description>
    <mitre><id>T1070</id></mitre>
    <group>windows_log_tampering,anti_forensics,</group>
  </rule>

  <!-- 4720: new user account created -->
  <rule id="100302" level="11">
    <if_sid>4720</if_sid>
    <description>User account $(TargetUserName) created by $(SubjectUserName)</description>
    <mitre><id>T1136</id></mitre>
    <group>windows_account_change,</group>
  </rule>

  <!-- 4698: scheduled task created -->
  <rule id="100303" level="10">
    <if_sid>4698</if_sid>
    <description>Scheduled task $(TaskName) created by $(SubjectUserName)</description>
    <mitre><id>T1053.005</id></mitre>
    <group>windows_persistence,</group>
  </rule>

  <!-- 7045: service installed -->
  <rule id="100304" level="11">
    <if_sid>7045</if_sid>
    <description>Service $(ServiceName) installed on $(SystemName)</description>
    <mitre><id>T1543.003</id></mitre>
    <group>windows_persistence,</group>
  </rule>

  <!-- 4719: audit policy changed. Someone is disabling your visibility. -->
  <rule id="100305" level="12">
    <if_sid>4719</if_sid>
    <description>Audit policy modified on $(SystemName) by $(SubjectUserName)</description>
    <mitre><id>T1562.002</id></mitre>
    <group>windows_defense_evasion,</group>
  </rule>

  <!-- 4740: account lockout. Signal, not noise. -->
  <rule id="100306" level="8">
    <if_sid>4740</if_sid>
    <description>Account $(TargetUserName) locked out on $(SystemName)</description>
    <group>windows_authentication,lockout,</group>
  </rule>

  <!-- Correlation: a lockout followed by SUCCESS from the same source.
       This is the rule that separates a typo from a real intrusion. -->
```

```
  <rule id="100307" level="13">
    <if_mit sid="100306">0,6,</if_mit>
    <same_source_ip />
    <description>$(TargetUserName) logged on after a lockout - possible credential
compromise</description>
    <mitre><id>T1110</id></mitre>
    <group>windows_correlation,credential_attack,</group>
  </rule>

</group>
```

> **Why rule 100307 is the best rule in this document**
>
> Every individual event it depends on is low-severity and mostly benign. Someone mistyping a password five times is not an incident. But *lockout followed by success from the same source* means the guessing eventually worked — and no amount of watching the individual events would have shown you that. This is what correlation means, and it is the difference between a SIEM that shows alerts and one that shows incidents.

P7.3 Test each rule before you trust it

1 Validate the XML

```
sudo /var/ossec/bin/wazuh-logtest -t
```

2 Test against a real event line

```
sudo /var/ossec/bin/wazuh-logtest
# paste the actual Windows log line that should trigger it
```

3 Restart, then refresh the agent config

```
sudo systemctl restart wazuh-manager
sudo /var/ossec/bin/agent_control -R 002
```

4 Trigger it for real, from Part P5

```
# e.g. re-run the 4732 test from P5.6
```

5 Confirm in the dashboard

```
rule.id:100300
agent.name:"winsrv-01" AND rule.level:>=10
```

If it does not appear, check the agent received the config (`agent_control -R`) before looking anywhere else.

P7.4 Saved searches worth having

```
rule.groups:windows_privilege_change
rule.groups:anti_forensics
rule.groups:windows_correlation
rule.groups:windows_persistence
data.win.system.eventID:1102
data.win.system.eventID:4719
rule.level:>=10 AND agent.name:"winsrv-01"
location:win_eventchannel AND rule.level:>=9
data.win.system.eventID:1 AND data.win.eventdata.CommandLine:*powershell*
```

Name them and bookmark them, and they become your daily workflow — the same habit from Volume 1 Part 6.2, applied to Windows.

P7.5 A dashboard for the Windows host

Dev Tools → Custom Dashboards. Build one panel per detection category, filtered to `agent.name:"winsrv-01"`, sorted by level descending. You now have a single screen answering "is anything wrong with our server?" — which is the entire deliverable a SIEM exists to provide.

P8 The writeup template

Stop at this point and the last day was an afternoon of clicking. Do this part and it becomes a portfolio piece. Structure, with the real content filled in:

```
## Windows Server 2022 Hardening & Detection Validation

**Scope:** WinServer (10.0.0.50), VirtualBox isolated lab network, 30 Sep 2026.
Authorised by: self (sole owner of the lab hardware and software).

### 1. Objective
Reduce the attack surface of a default Windows Server 2022 install and prove the
SIEM detects the relevant activity classes.

### 2. Initial state
- Open ports: [from nmap]
- SMBv1: [enabled/disabled]
- SMB signing: [state]
- Minimum password length: [n]
- Account lockout: [state]
- Guest account: [state]
- SCA pass rate: [n]% ([x] of [y] checks)
- Wazuh alerts from this host: [n]

### 3. Findings
| # | Finding | Risk | Control applied | Evidence |
|---|---------|------|-----------------|----------|
| 1 | SMBv1 enabled | High | Disabled feature | nmap after-output |
| 2 | No lockout policy | Medium | 5 tries / 15 min | net accounts |
| 3 | Group membership not audited | High | Audit subcategory enabled | event 4732 alert |

Explain WHY each finding matters, in one sentence. This is the part that shows judgement.

### 4. Changes applied
[the commands, grouped by category, with a short rationale each]

### 5. Post-change state
[the before/after table from P4.9]

### 6. Detection validation
[the matrix from P5.11, completed]

For each row where detection was achieved, include:
- what you did
- the Windows event ID generated
- the Wazuh rule that fired
- a screenshot of the alert

### 7. Gaps and limitations
Be honest and specific. This is the most impressive section:
- "5145 events are generated but no rule correlates them - rule needed"
- "Process creation auditing disabled due to volume; Sysmon used instead"
- "No network-level detection - this lab has no NDR"

### 8. What I would do next
[see Part P9]
```

Section 7 is why you will get the interview

Most people who show a hardening project show only the wins. Documenting what you found broken, what you chose not to fix, and why — demonstrates exactly the professional judgement an employer is screening for. It also shows you can think past the task.

P9 Project roadmap — what to build next

You asked what to do after this. Here is a ranked list, best value first, sized for a beginner working evenings.

#	Project	Why, and what it teaches
1	**Promote WinServer to a domain controller**	Install AD DS, create users and OUs, apply Group Policy centrally. This is the single biggest jump in realism and it is where most real Windows security happens. You will learn Kerberos, group policy, logon types, SYSVOL and replication. Everything else on this list is easier once you have done this.
2	**Suricata on SIEM-Ubuntu**	Wazuh sees what hosts *report*. Suricata sees what actually crosses the wire — including things no host reports. Install it, write a few rules, ship `eve.json` into Wazuh with a Filebeat input. You go from host-based to network detection, which is a major capability gap in most junior CVs.
3	**Enroll WindowsClient, add a third VM**	Two or three agents makes agent groups, labels and multi-agent searches genuinely necessary instead of theoretical. Cheap to do, and it immediately makes your dashboards useful.
4	**Sigma rules**	A portable detection format. Write one for your 4732 rule, run it in Wazuh, and you have a skill that transfers between every SIEM vendor. Strong portfolio signal for modest effort.
5	**Network segmentation with a firewall VM**	Put pfSense or OPNsense between segments, then show Wazuh alerts changing when you move a machine across. Teaches segmentation as a control and gives you packet-level visibility.
6	**osquery**	SQL-query your endpoints. Pairs well with Wazuh as an independent second opinion on process state.
7	**Configuration as code**	Move agent configuration and the hardening script out of the manager into Ansible. Teaches infrastructure as code — directly transferable professional skill, and you already know Linux well enough.
8	**Theory to pair with the lab**	TryHackMe rooms on AD, Windows defence and blue team, plus PortSwigger for understanding the techniques you are detecting. Your lab gives you somewhere to practise; these give you the vocabulary an interviewer will use.
9	**Publish it**	A GitHub repo with your `local_rules.xml`, your FIM config, `harden-server.ps1`, your SCA before/after, and the Part P8 writeup. Then a short post about it. **This is the highest-return hour you will spend.**

P9.1 Turn this lab into a script

The hardening in Part P4 is a perfect candidate for automation. Package it as `harden-server.ps1` that runs on any fresh Windows Server VM, and you have demonstrated repeatability — which is what an employer actually means by "automation".

```
# harden-server.ps1 - run as Administrator
param([string]$LabIP = "10.0.0.50")

Write-Host "=== Password and account policy ==="
net accounts /minpwlen:14 /minpwage:1 /maxpwage:60 /lengthenable:yes | Out-Null
net accounts /lockoutthreshold:5 /lockoutduration:15 /lockoutobservation:30 | Out-Null
net user guest /active:no | Out-Null

Write-Host "=== SMB hardening ==="
if (Get-WindowsOptionalFeature -Online -FeatureName SMB1Protocol) {
  Disable-WindowsOptionalFeature -Online -FeatureName SMB1Protocol -NoRestart | Out-Null
}
Set-SmbServerConfiguration -RequireSecuritySignature $true -EnableSecuritySignature $true -Force
Set-SmbServerConfiguration -EncryptData $true -Force
Set-SmbServerConfiguration -EnableAuthNullUser $false -Force

Write-Host "=== Null session restriction ==="
$lsa = "HKLM:\SYSTEM\CurrentControlSet\Control\Lsa"
Set-ItemProperty $lsa -Name RestrictAnonymous         -Value 1
Set-ItemProperty $lsa -Name RestrictAnonymousSAM      -Value 1
Set-ItemProperty $lsa -Name restrictnullsessions      -Value 1
Set-ItemProperty $lsa -Name EveryoneIncludesAnonymous -Value 0

Write-Host "=== LSA protection and Credential Guard ==="
Set-ItemProperty $lsa -Name RunAsPPL -Value 1
Enable-WindowsOptionalFeature -Online -FeatureName CredentialGuard -All | Out-Null

Write-Host "=== Audit policy ==="
auditpol /backup /file:"C:\Windows\Temp\audit-backup.csv"
auditpol /set /subcategory:"Logon" /success:enable /failure:enable | Out-Null
auditpol /set /subcategory:"Special Logon" /success:enable | Out-Null
auditpol /set /subcategory:"Logon (Service Package)" /success:enable | Out-Null
auditpol /set /subcategory:"Credential Validation" /success:enable /failure:enable | Out-Null
auditpol /set /subcategory:"Account Lockout" /failure:enable | Out-Null
auditpol /set /subcategory:"Security Group Management" /success:enable | Out-Null
auditpol /set /subcategory:"User Account Management" /success:enable | Out-Null
auditpol /set /subcategory:"Audit Policy Change" /success:enable | Out-Null
auditpol /set /subcategory:"Detailed File Share" /success:enable | Out-Null
Set-ItemProperty "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" `
  -Name AuditProcessCreationIncludeCmdLine_Enabled -Value 1

Write-Host "=== PowerShell logging ==="
$ps = "HKLM:\Software\Policies\Microsoft\Windows\PowerShell"
New-Item "$ps\ScriptBlockLogging" -Force | Out-Null
Set-ItemProperty "$ps\ScriptBlockLogging" -Name EnableScriptBlockLogging -Value 1

Write-Host "=== Firewall ==="
Set-NetFirewallProfile -Profile Domain,Public,Private -Enabled True `
  -DefaultInboundAction Block -DefaultOutboundAction Allow
New-NetFirewallRule -DisplayName "Wazuh Agent 1514" -Direction Inbound `
  -Action Allow -Protocol TCP -LocalPort 1514 -Profile Any -ErrorAction SilentlyContinue | Out-Null

Write-Host "=== Networking ==="
Set-NetIPInterface -InterfaceAlias "Ethernet" -Dhcp Disabled -ErrorAction SilentlyContinue
if (-not (Get-NetIPAddress -InterfaceAlias "Ethernet" -AddressFamily IPv4 `
           -ErrorAction SilentlyContinue | Where-Object IPAddress -eq $LabIP)) {
  New-NetIPAddress -InterfaceAlias "Ethernet" -IPAddress $LabIP -PrefixLength 24
}

Write-Host ""
```

```
Write-Host "=== Verification - compare against your baseline ===" -ForegroundColor Cyan
Get-SmbServerConfiguration | Format-List EnableSMB1Protocol, RequireSecuritySignature, EncryptData
auditpol /get /subcategory:"Logon"
Get-NetFirewallProfile | Format-Table Name, Enabled, DefaultInboundAction -AutoSize
Write-Host "Reboot required for Credential Guard and LSA protection to fully apply."
```

> **Run it against a snapshot, not against your working server**
>
> This is exactly the kind of thing to test on a throwaway VM. That is what snapshots are for. And notice the script ends by printing its own verification — automation that checks its own work is the difference between a script and a tool.

P9.2 Where you actually are now

Two documents ago you had a multi-layer network fault and a partly installed SIEM. You now have a three-node Wazuh stack with TLS, a live agent fleet, a hardened Windows Server, an audit policy tuned for detection, custom correlation rules, and a validation matrix you filled in yourself.

That last item is the one to be proudest of. Anyone can install a product. Knowing exactly which of your detections fire, which do not, and why — that is the professional skill, and it is the thing you will be able to talk about in an interview for an hour without running out of material.

> **Document conventions**
>
> Every IP, port, MAC and build number in this guide was verified against your actual VMs on 30 September 2026. Values marked *planned*, *optional* or *replace with your adapter name* are the ones you may need to adjust. Several Wazuh field names differ slightly between versions, which is why P7.1 makes you confirm them with `wazuh-logtest` rather than trusting any document — including this one.

Appendix A Windows security event ID reference

Learn these twenty and you can read a Windows server. Print this page.

A.1 Authentication (Security log)

ID	Event	Why it matters
4616	System time changed	Timestamp manipulation — breaks every log correlation afterwards
4624	Successful logon	The foundation event. Read `LogonType` to know how.
4625	Failed logon	Authentication attempts that failed
4634 / 4635	Logoff / logon session terminated	Session lifecycle
4647	User initiated logoff	Clean shutdown initiated by the user
4648	Logon with explicit credentials	How `runas` works; a way to hide the real account
4672	Special privileges assigned to new logon	A new session holding admin rights
4776	Credential validation (NTLM)	NTLM authentication attempts
4740	Account locked out	Something tried repeatedly and failed

A.2 Account and group management (Security log)

ID	Event	Why it matters
4720	User account created	Persistence: a new account
4722	User account enabled	A disabled account turned back on
4724	Attempt to reset a password	Possible account takeover
4725	Attempt to delete a user account	Cleanup after use
4726	User account deleted	Cleanup
4728	Member added to a global security group	Privilege change
4729 / 4731	Member added to a local / universal group	Privilege change
4732	**Member added to a privileged group**	**The escalation event. Alert on this.**
4738 / 4742	Account renamed / changed	Possible evasion of name-based rules
4756	Member added to a universal security group	Privilege change (domain)

A.3 Object access, processes and persistence

ID	Event	Why it matters
`4688`	Process created	Everything that runs. Highest volume — needs cmdline enabled to be useful
`4697`	Service installed (Security log)	Persistence via a service
`4698 / 4699`	Scheduled task created / deleted	The most common persistence mechanism
`4719`	Audit policy modified	Someone is disabling your monitoring
`5140`	Share accessed	Data access
`5145`	Detailed object access	Precise file access. Noisy but valuable.
`7045`	Service installed (**System log**)	Persistence. Note: different log from 4697.

A.4 Defence evasion and PowerShell

ID	Event	Why it matters
`1102`	Audit log cleared (**Security log**)	Covering tracks. Alert at high severity, always.
`4103`	PowerShell module logging	Module use recorded
`4104`	**PowerShell script block**	**The script itself, even when obfuscated. Very high value.**
`5156`	Firewall allowed a connection	Requires *Audit Filtering Platform Connection* — very high volume
`5157`	Firewall blocked a connection	Same caveat; useful for tuning

A.5 Sysmon event IDs (Microsoft-Windows-Sysmon/Operational)

ID	Event	Why it matters
`1`	Process creation	Like 4688 but leaner and always with the full command line
`3`	Network connection	Outbound connections, including to unusual ports
`7`	Image loaded	DLL injection and sideloading detection
`8`	CreateRemoteThread	Classic code injection technique
`10`	Process accessed	Credential access patterns
`11`	File created	Dropping files
`12 / 13 / 14`	Registry created / value set / renamed	Run keys, service changes, Defender tampering
`22`	DNS query	Command-and-control beaconing

Appendix B Command cheat sheet

B.1 On Kali (audit and observation)

```
ip -br a; ip neigh                              # who am I, who is next to me
ping -c3 10.0.0.50
sudo nmap -Pn -sV -sC -p- --open 10.0.0.50
sudo nmap -Pn -p 445 --script smb-protocols,smb2-security-mode,smb-enum-shares 10.0.0.50
smbclient -L //10.0.0.50 -N
smbmap -H 10.0.0.50
netexec smb 10.0.0.50 --pass-pol
ssh labadmin@10.0.0.50
```

B.2 On WinServer (PowerShell, Administrator)

```
net accounts                                    # password + lockout policy
net user <name> /active:no
Get-LocalUser | Format-Table Name, Enabled -AutoSize
Get-LocalGroupMember -Group "Administrators"
Get-SmbServerConfiguration | Format-List *
Set-SmbServerConfiguration -RequireSecuritySignature $true -Force
Get-NetFirewallProfile | Format-Table Name, Enabled, DefaultInboundAction -AutoSize
Get-NetFirewallRule -DisplayName "Wazuh*"
auditpol /backup /file:C:\Windows\Temp\audit-backup.csv
auditpol /restore /file:C:\Windows\Temp\audit-backup.csv
auditpol /get /subcategory:"Logon"
Get-MpComputerStatus | Format-List *
Get-Service Wazuh | Format-Table Name, Status, StartType -AutoSize
Test-NetConnection -ComputerName 10.0.0.40 -Port 1514
wevtutil qe Security /c:10 /rd:true /f:text
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 10
```

B.3 On SIEM-Ubuntu (Wazuh manager)

```
sudo systemctl status wazuh-manager wazuh-indexer wazuh-dashboard filebeat
sudo /var/ossec/bin/agent_control -l
sudo /var/ossec/bin/agent_control -R 002
sudo /var/ossec/bin/wazuh-logtest -t
sudo /var/ossec/bin/wazuh-logtest
sudo tail -f /var/ossec/logs/alerts/alerts.json | python3 -m json.tool
curl -k -u admin:<MANAGER_PASSWORD> \
  https://127.0.0.1:9200/_cluster/health?pretty
```

B.4 On Windows host (VirtualBox)

```
VBoxManage showvminfo "WinServer" | Select-String "NIC|Rule|DVD"
VBoxManage modifyvm "WinServer" --medium none
VBoxManage controlvm "WinServer" snapshot take "winserver-audited-30sep"
VBoxManage guestcontrol "WinServer" run --username .\labadmin --password <pw> `
  --exe "C:\Windows\System32\cmd.exe" -- /c "whoami"
ssh -p 2222 ubuntu@127.0.0.1          # SIEM-Ubuntu
```

B.5 The three questions, every time

> **Which machine?** Agent name and IP. **What exactly happened?** Rule description and level. **How do I prove it?** The raw event fields.
>
> If you cannot answer all three from the alert alone, you do not understand the alert yet — and neither will anyone reviewing your work.